CAMPUS LIFE

Assignment 5

Advanced JavaScript and DOM Manipulation

Astana IT University - Web Technologies

Submission details	Value
Team	Campus Life
Members	Yessetaiuly Inal; Nurkeldi Baizhigit
Group	SE-2508
Date	28 October 2026
Repository	https://github.com/inalesetaiuly/web-technology_project
Website	https://inalesetaiuly.github.io/web-technology_project/
Local browser verification	47 automated checks passed in Chrome

Objective

Extend the existing student guide with meaningful DOM manipulation, event handling, reusable data structures, callbacks, sound and event-triggered animation. Preserve the previous assignment features and JavaScript validation rules.

The report uses real screenshots captured from the implemented site in Chrome, together with screenshots of source excerpts. Form data is demonstration-only and is not transmitted or stored. Only the theme preference is persisted.

1. Requirements and selected options

The assignment offers alternatives. We selected Day/Night theme for dynamic styling, personalized greeting for content manipulation, and a multi-step form for the callback requirement. The remaining required concepts are implemented explicitly.

Criterion	Implementation	Weight
DOM manipulation and styling	Theme attribute/classes, greeting text, generated cards and ARIA states	10%
Event handling	Clicks, submit, keydown, callbacks and switch	10%
Data structures and functions	Catalog object/methods, array, for...of, filter(), modular functions	10%
Sound effects	User-triggered two-tone Web Audio notification; overlap guard	5%
Animations	Greeting, cards, step changes and theme transitions	5%
Clean code and structure	Separate HTML, CSS and external JavaScript files	5%
UX and engagement	Student-guide theme, keyboard access, responsive layout, reduced motion	5%
Defense	Demonstration and explanation guide; performed by students	40%
Report	Objective, implementation steps, code/UI screenshots and reflection	10%

File structure

index.html: greeting, clock, background and sound. resources.html: generated resource catalog and FAQ. contact.html: multi-step form and subscription dialog. about.html: team and carousel. css/style.css: visual rules and themes. js/site.js: shared validation and Assignment 4. js/assignment5.js: new components. tests/assignment5.cjs: repeatable browser checks.

2. Day and night themes

1. Add a theme button to every navbar. 2. Read the saved preference, or use the system color scheme. 3. Change data-bs-theme on the root HTML element. 4. Update button text and aria-pressed. 5. Save the selected theme in localStorage; handle unavailable storage without breaking the toggle.

CSS variables cover page surfaces, text, cards, tables, forms, dialog, links and focus indicators. The old background-color control applies to day mode only, so it cannot override the dark background.

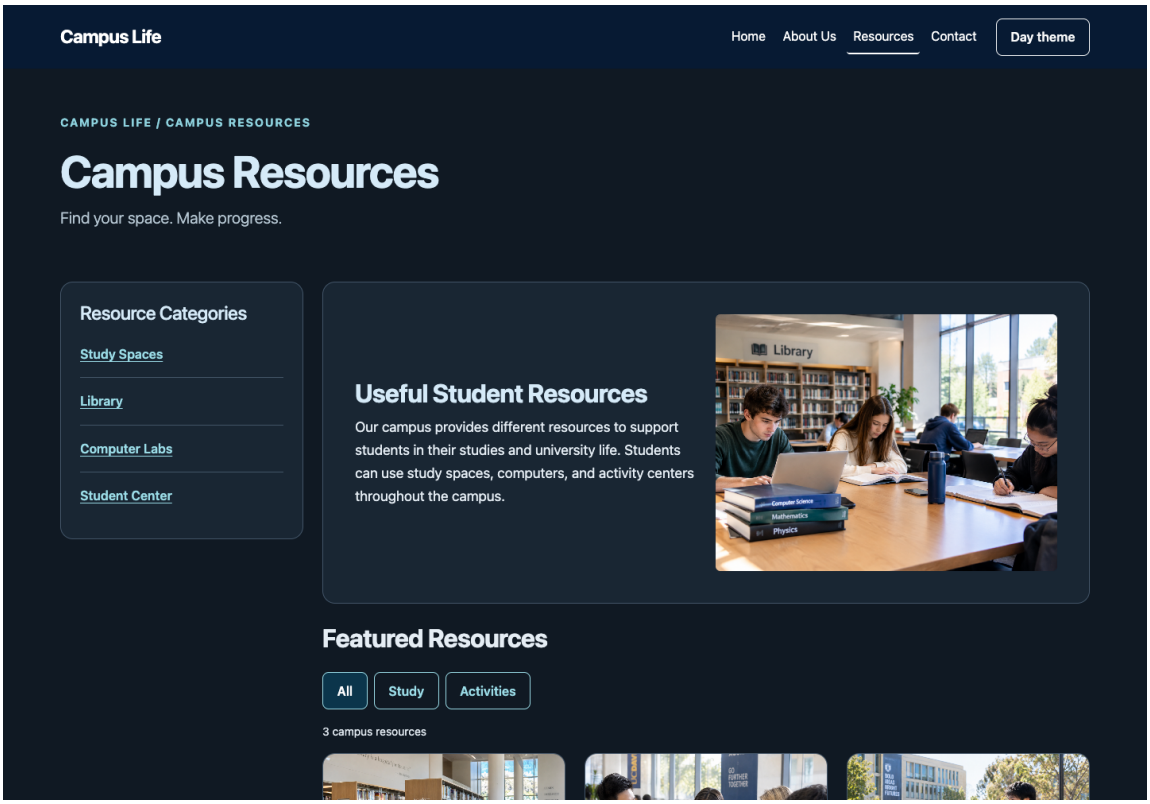


Figure 1. Actual Resources page in night theme.

```
function applyTheme(theme) {
    const dark = theme === 'dark';
    document.documentElement.dataset.bsTheme = dark ? 'dark' : 'light';
    button.textContent = dark ? 'Day theme' : 'Night theme';
    button.setAttribute('aria-pressed', String(dark));
    button.setAttribute('aria-label', `Switch to ${dark ? 'day' : 'night'} theme`);
}
applyTheme(['light', 'dark'].includes(preference) ? preference : (media.matches ? 'dark' : 'light'));
button.hidden = false;
```

Figure 2. Source screenshot: theme application and persistence.

3. Personalized greeting and DOM

1. Select the greeting form and output by ID. 2. Listen for submit and prevent navigation. 3. Trim the name and validate non-empty input and a maximum length of 60 in JavaScript. 4. Write the welcome message using textContent. 5. Animate the updated message.

Enter and the button use the same handler. HTML-looking input is shown as literal text rather than inserted as markup. The name is not persisted. Empty/whitespace-only input is rejected even if the HTML required attribute is removed.

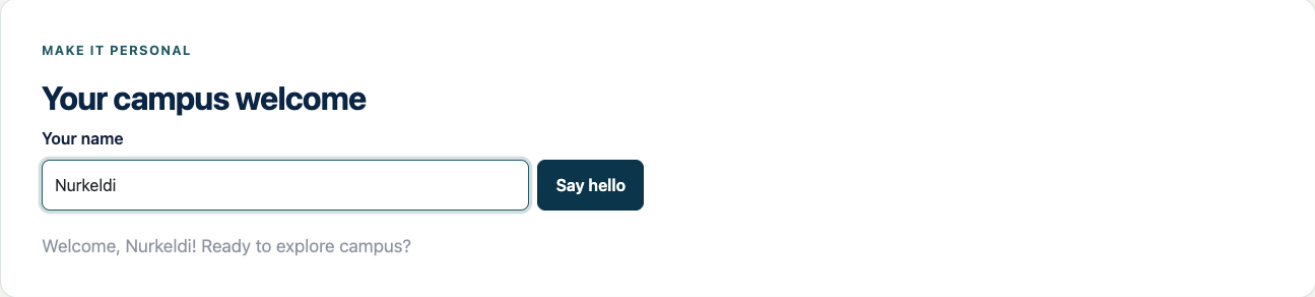


Figure 3. A personalized greeting on the Home page.

```
        const field = form.querySelector('#greeting-name');
        const name = field && !field.disabled ? field.value.trim() : '';
        const message = !name ? 'Please enter your name.' : name.length > 60 ? 'Use no more than 60
characters.' : '';
        error.textContent = message;
        error.hidden = !message;
        if (field) {
            showFieldError(field, message);
            if (message) field.focus();
        }
        // textContent treats any HTML-looking input as text, never as markup.
        result.textContent = message ? 'Welcome to Campus Life!' : `Welcome, ${name}! Ready to explore campus?
`;
```

Figure 4. Source screenshot: input validation and safe DOM text update.

4. Keyboard events and navigation

1. Attach keydown to the navbar link list. 2. Find the currently focused link. 3. Use switch(event.key) to choose a direction. 4. Calculate the next index using modulo so movement wraps. 5. Call focus() and prevent arrow-key scrolling.

ArrowRight/ArrowDown move forward, ArrowLeft/ArrowUp move back, and Home/End choose the boundary items. Tab and Enter retain their native behavior. The mobile navbar must be opened before navigating its links.

```
function setupKeyboardNavigation() {
    const menu = document.querySelector('#mainNavbar .navbar-nav');
    if (!menu) return;
    menu.addEventListener('keydown', event => {
        if (event.altKey || event.ctrlKey || event.metaKey) return;
        const links = Array.from(menu.querySelectorAll('.nav-link')).filter(link =>
link.getClientRects().length);
        const index = links.indexOf(document.activeElement);
        if (index < 0) return;
        let next;
        switch (event.key) {
            case 'ArrowRight': case 'ArrowDown': next = (index + 1) % links.length; break;
            case 'ArrowLeft': case 'ArrowUp': next = (index - 1 + links.length) % links.length; break;
            case 'Home': next = 0; break;
            case 'End': next = links.length - 1; break;
            default: return;
        }
        event.preventDefault();
        links[next].focus();
    });
}
```

Figure 5. Source screenshot: switch-based arrow navigation.

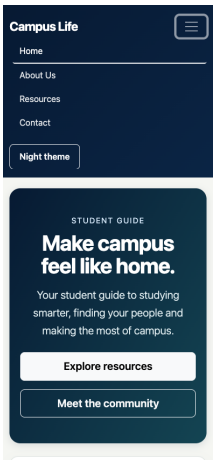


Figure 6. Actual Home page at a 390px viewport.

5. Multi-step form and callback

1. Define three step objects with their field IDs. 2. Create a controller that stores the current index in a closure. 3. Pass onStepChange as a callback. 4. Next validates the current fields before calling goTo(). 5. Back changes the index without clearing inputs. 6. The callback shows the correct groups, updates progress and focuses the step label.

The permitted multi-step option is used instead of an asynchronous POST. No backend or real account creation is claimed. Enter advances an intermediate step, while the last submit checks the complete JavaScript schema.

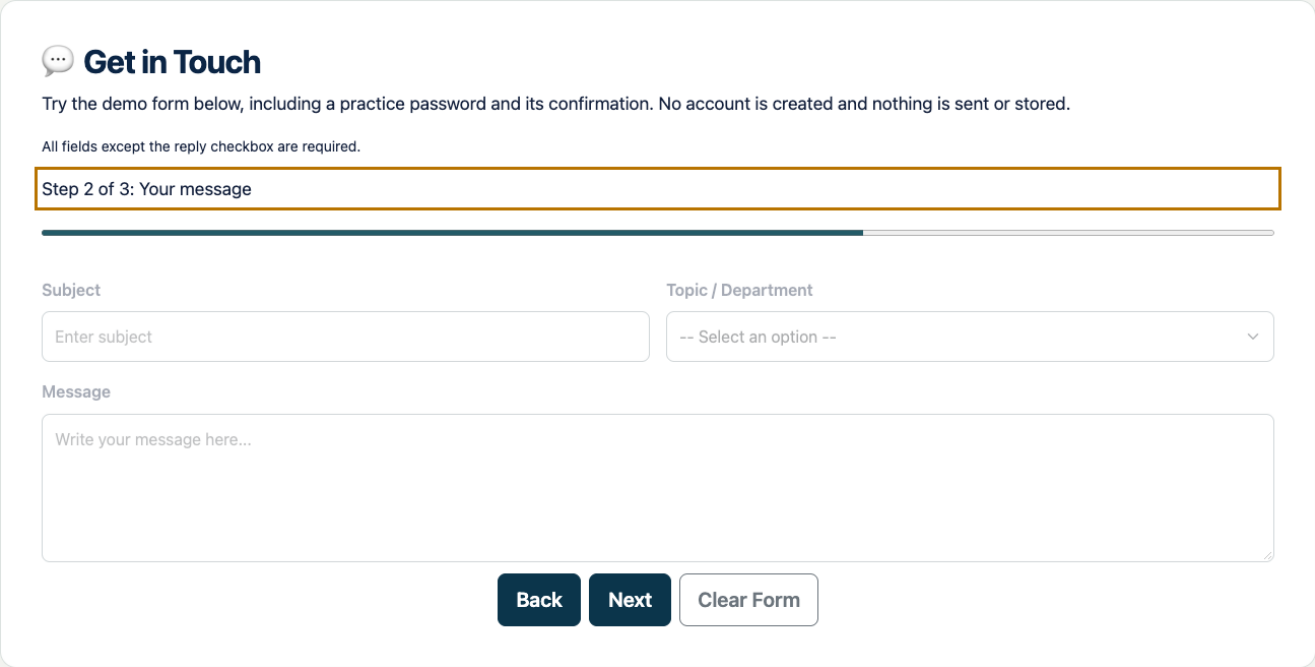


Figure 7. Step 2 of 3 with Back/Next and a progress indicator.

```
function createStepController(total, onStepChange) {
    let current = 0;
    return {
        get current() { return current; },
        goTo(index, focus = true) {
            if (!Number.isInteger(index) || index < 0 || index >= total) return;
            current = index;
            onStepChange(current, focus);
        }
    };
}
```

Figure 8. Source screenshot: a controller invoking its callback.

6. Validation and failure cases

Rules remain in contactRules rather than depending on required, type, minlength or data-match attributes. Every submission re-reads mandatory controls. Missing, duplicated or disabled controls block success. Earlier invalid fields cause their step to open before focus moves to the error.

The final check rejects incomplete input, malformed email, invalid category, short passwords and mismatched confirmation. Successful validation clears practice passwords. Reset clears errors and restores step 1.

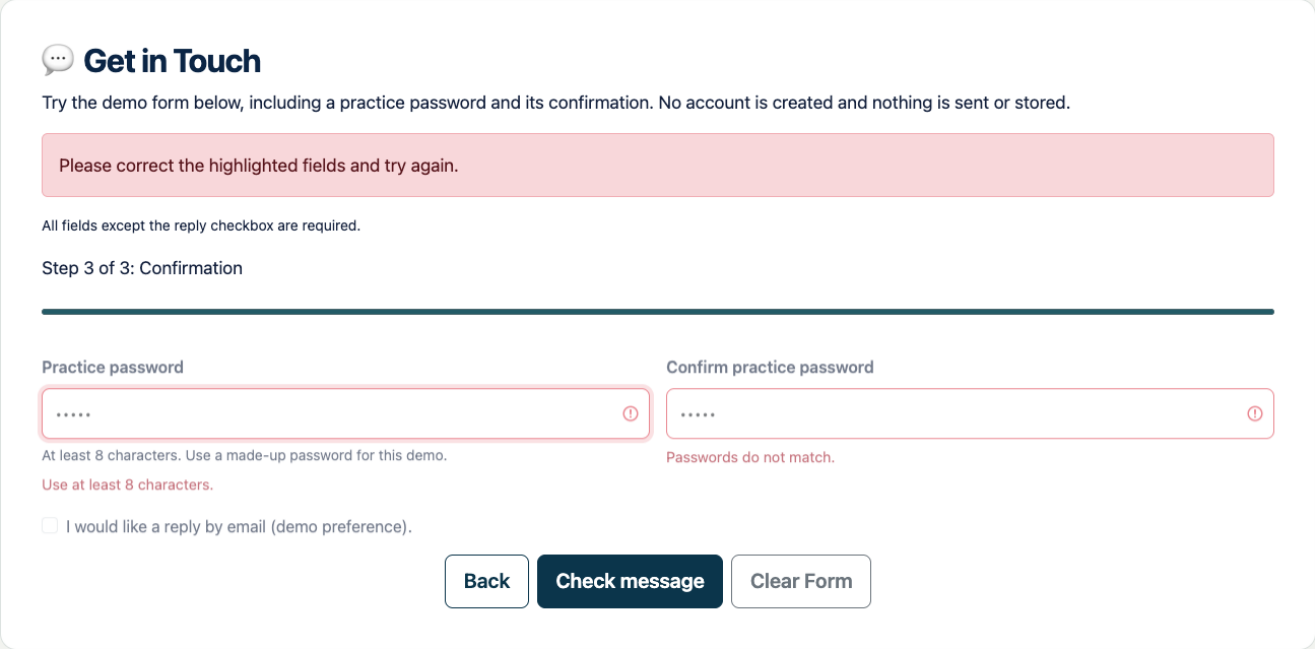


Figure 9. Actual errors for a short password and different confirmation.

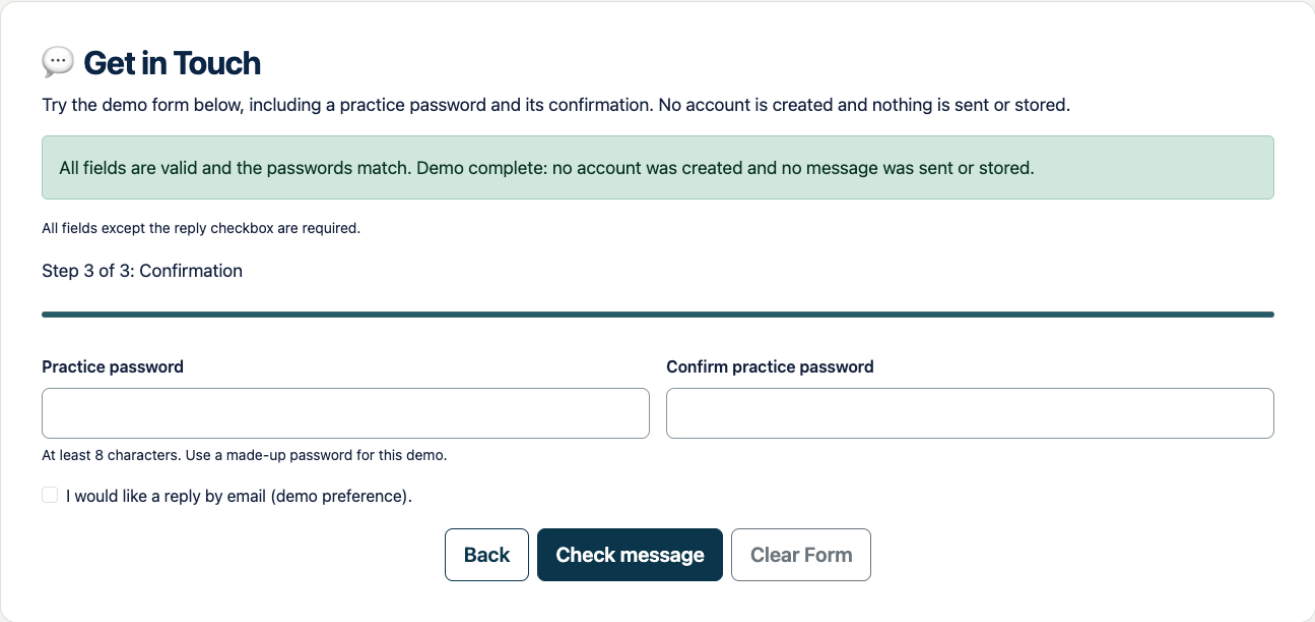


Figure 10. Successful demo validation; no real data is sent.

7. Objects, arrays, loops and filtering

1. Store resources as objects in catalog.items. 2. Use the filterByCategory() method and switch to select All, Study or Activities. 3. Use filter() with a predicate callback for category matching. 4. Iterate selected objects with for...of. 5. Create cards with createElement(), assemble a DocumentFragment and insert it with replaceChildren().

The count updates with textContent and category buttons update aria-pressed. All shows 3 cards, Study shows 2 and Activities shows 1. Sidebar links restore all items before jumping to a resource, preserving existing navigation.

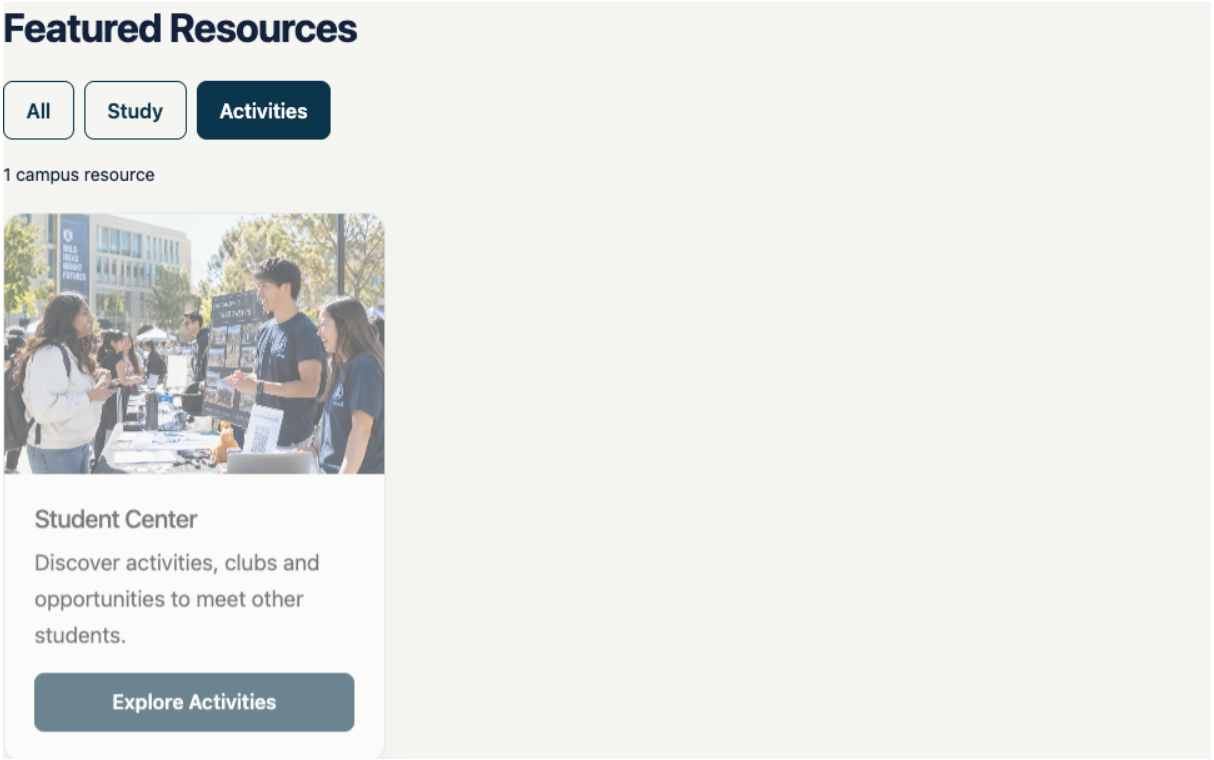


Figure 11. Actual catalog filtered to Activities with one result.

```
filterByCategory(category) {
    switch (category) {
        case 'study': case 'activities': return this.items.filter(item => item.category === category);
        case 'all': default: return this.items;
    }
},
```

Figure 12. Source screenshot: a method, switch and higher-order filter().

8. Sound, animations and usability

A click creates/resumes an AudioContext. An oscillator generates two short tones (660 and 880 Hz), and a GainNode fades amplitude. A playing flag and disabled button prevent overlapping sounds. The nodes disconnect after onended; failure displays a status message.

animateEntrance() uses Element.animate() on greeting changes, filtered cards and visible form steps. It cancels prior animations on the target. CSS transitions smooth theme colors. The prefers-reduced-motion preference disables optional motion. Sounds are never played automatically.

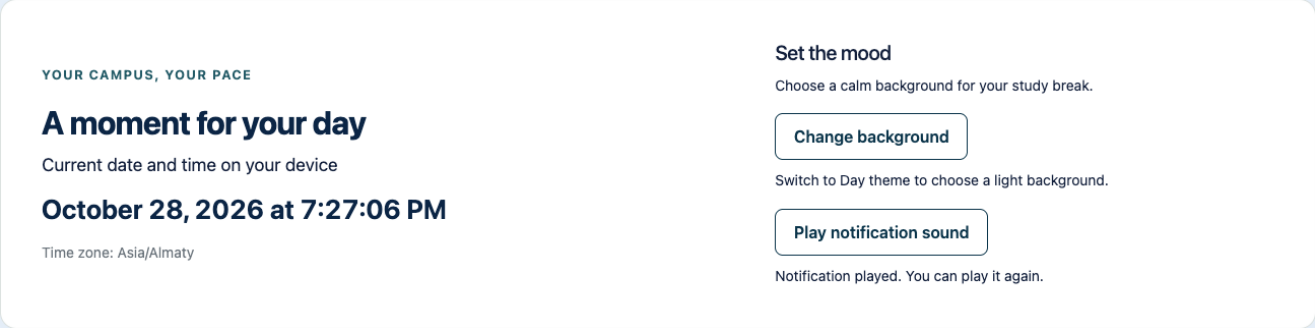


Figure 13. Actual clock, background control and notification button.

```
const start = context.currentTime;
oscillator.frequency.setValueAtTime(660, start);
oscillator.frequency.setValueAtTime(880, start + 0.12);
gain.gain.setValueAtTime(0, start);
gain.gain.linearRampToValueAtTime(0.12, start + 0.02);
gain.gain.linearRampToValueAtTime(0, start + 0.3);
status.textContent = 'Playing campus notification…';
await new Promise(resolve => {
    oscillator.onended = resolve;
    oscillator.start(start);
    oscillator.stop(start + 0.32);
});
```

Figure 14. Source screenshot: oscillator timing and gain envelope.

The automated check confirms oscillator start and completion and the overlap guard. Actual audibility depends on system volume and audio permissions; confirm it with speakers before the live defense.

9. Final test results

Test date: 28 October 2026. Environment: Chrome through Playwright, local HTTP server. Result: 47 checks passed; 0 uncaught JavaScript errors and 0 failed HTTP asset responses.

Area	Verified scenarios
Theme	Toggle, ARIA state, reload persistence, cross-page persistence and return to day mode
Greeting	Blank/whitespace, Enter submission and HTML-looking input rendered safely
Keyboard	Arrow wrap, Home/End and collapsed mobile navigation
Form	Next/Back, retained input, final success, password clearing and reset
Validation regression	Removed HTML constraints; all 7 mandatory fields individually deleted; earlier email tampering
Catalog	3/2/1 category counts, pressed state and restoration by sidebar links
Sound / motion	Oscillator completion, no overlap, unsupported audio, reduced-motion animation
Previous features	Clock tick, day background, FAQ keyboard toggle, popup validation/close, nine-slide carousel
Responsive layout	All 4 pages at 390px, 768px and 1440px with no horizontal page overflow

Reproduce the test

Install Playwright in a development environment and run node tests/assignment5.cjs. CHROME_EXECUTABLE can select an existing Chrome binary. The suite starts a local server, executes browser checks and captures evidence. No development tools are needed to open the website itself.

Full named results: output/assignment5/test-results.json. Browser evidence: output/assignment5/evidence/. The test covers behavior; personal understanding and the oral defense cannot be automated.

10. Reflection and defense preparation

Challenges and solutions

Theme consistency: fixed colors from earlier work needed coordinated dark overrides. The solution uses theme variables and Bootstrap color-mode support rather than changing only the page background.

Form integration: hidden steps must not bypass mandatory checks. The solution keeps all JavaScript rules active at final submission and reveals the relevant step on validation errors. Existing fields remain enabled, preserving their values.

Filtering and anchors: generated cards can disappear under a filter. Sidebar/hash navigation restores the full catalog before scrolling. Sound overlap is prevented by explicit playback state and cleanup.

What the team learned

DOM methods connect JavaScript state to visible content. A callback lets a controller reuse presentation logic. Objects organize related data and methods; arrays and filter() provide predictable selections. Keyboard support, reduced motion and error handling are part of the feature implementation, not separate decoration.

Demonstration sequence

1. Switch to night theme, navigate to another page, reload, then return to day.

2. Submit an empty greeting, then enter a name; navigate the navbar with arrow keys.

3. Play the notification; explain the overlap guard and audio cleanup.

4. Filter resources to Study/Activities and explain the object, switch, loop and filter predicate.

5. Walk through form steps, go Back, demonstrate errors and successful validation; explain the callback.

6. Explain how JavaScript rules survive removed HTML constraints and why real services still need server validation.

Submission

Submit this PDF and the project ZIP with HTML, CSS, JavaScript and local assets. Include the deployed URL shown on the cover. Each team member must upload the work to the LMS before the course deadline and personally defend it. The 40% defense component depends on the students explaining and modifying the code themselves.